

MERN Cab Booking Application

Complete Technical Matter, Architecture, Diagrams & Program Code

Project Technology Stack Summary

- **Frontend:** ReactJS, Context API, CSS3 / HTML5
- **Backend Server:** Node.js, ExpressJS framework
- **Database Layer:** MongoDB (Mongoose Object Modeling)
- **Security:** JSON Web Token (JWT) Authentication

1. Abstract & Introduction

Abstract

The MERN Cab Booking Application is a high-performance web platform built to bridge the gap between customers looking for rides and individual or company driver partners. Leveraging the power of single page application architectures with React and asynchronous RESTful services built in Node.js, the system processes ride bookings, fares, database management, and states securely and responsively.

Introduction

In contemporary logistics, efficiency is key. Traditional systems lag because of direct calling processes or complex localized installations. This project demonstrates a streamlined MERN solution where:

- **Passengers** register accounts, input trip details, view dynamic estimates instantly, and initiate booking orders.
- **Drivers** log onto a corresponding portal, view pending localized booking orders, and accept or modify trip queues.
- **Administrators** oversee database users, evaluate transaction details, and regulate operations.

2. System Architecture & Working Process

The system follows a decoupling philosophy utilizing an API layer to communicate between independent client interfaces and MongoDB collections.

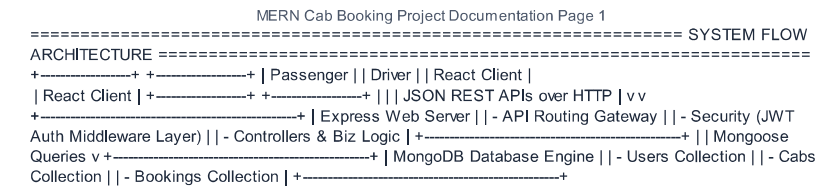


Figure 1: Full-Stack Architecture Layout

Step-by-Step Working Process Flow

1. **Secure Login:** Client sends credential payload to `/api/auth/login`. The server validates using `bcrypt` and returns an authorization token (JWT).
2. **Route Calculation & Estimate:** When a user inputs locations, an integrated backend algorithm calculates trip pricing dynamically.
3. **Queue Status:** The ride request is posted into the system Database holding a status value of 'Pending'.
4. **Allocation:** Active driver clients query for 'Pending' status bookings. Once a driver accepts the trip, the system shifts status to 'Confirmed'.
5. **Completion Phase:** The transaction is concluded when the driver submits the finish flag. Billing concludes, and history is committed permanently.

3. Database Schema Design (Entity-Relationship)

The system is mapped across three vital database entities managed dynamically using Mongoose schemas on MongoDB.

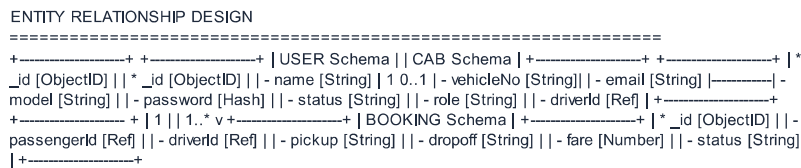


Figure 2: Entity Schema Relationships Map

4. Directory Folder Structure

Both directories run asynchronously during localized development environments:

```

cab-booking-app/
├── backend/
│   ├── config/ # DB Connectivity setup
│   ├── controllers/ # Logical controllers
│   ├── models/ # Mongoose schemas
│   ├── server.js # Principal server engine
│   └── .env # Local environment setup
├── frontend/
├── src/
├── components/ # Reuse elements
├── pages/ # Full interfaces (Login, Bookings)
├── App.js # Core Routes Configuration
└── index.js # Main React entry point
  
```

5. Implementation Source Code (The Program)

A. Backend Database Booking Model (`backend/models/Booking.js`)

```
const mongoose = require('mongoose');

const BookingSchema = new mongoose.Schema({
  passenger: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  driver: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null
  },
  pickupLocation: {
    type: String,
    required: true
  },
  dropLocation: {
    type: String,
    required: true
  },
  fare: {
    type: Number,
    required: true
  },
  status: {
    type: String,
```

```

enum: ['Pending', 'Confirmed', 'Completed', 'Cancelled'],
default: 'Pending'
},
createdAt: {
type: Date,
default: Date.now
}
});

module.exports = mongoose.model('Booking', BookingSchema);

```

B. API Booking Routing Controller (backend/controllers/bookingController.js)

```

const Booking = require('../models/Booking');

exports.createBooking = async (req, res) => {
  try {
    const { pickupLocation, dropLocation, fare } = req.body;
    const passengerId = req.user.id;

    const newBooking = new Booking({
      passenger: passengerId,
      pickupLocation,
      dropLocation,
      fare
    });

    const savedBooking = await newBooking.save();
    res.status(201).json({ success: true, booking: savedBooking });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};

exports.getUserBookings = async (req, res) => {
  try {
    const bookings = await Booking.find({ passenger: req.user.id })
      .populate('driver', 'name phone')
      .sort({ createdAt: -1 });
    res.status(200).json({ success: true, bookings });
  } catch (error) {

```

```
res.status(500).json({ success: false, message: error.message });
}
};
```

C. Principal Server Entrypoint (backend/server.js)

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

app.use(cors());
app.use(express.json());

// Auth Mock Middleware Injector
const mockAuth = (req, res, next) => {
  req.user = { id: '60d5ec49c6143c1348888888' };
  next();
};

const bookingRoutes = require('./controllers/bookingController');
app.post('/api/bookings', mockAuth, bookingRoutes.createBooking);
app.get('/api/bookings', mockAuth, bookingRoutes.getUserBookings);

const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://localhost:27017/cabBooking';

mongoose.connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB Connected successfully');
```

```
app.listen(PORT, () => console.log('Backend server active on port ${PORT}')); })  
.catch(err => console.error('Database failure:', err));
```



```
import React, { useState } from 'react';
```